

GitHub Copilot in a Codespace

Ask_GPT-5mini

Q: I am a student who is a first-time learner of Python and its packages. Please provide an at-a-glance summary of how pandas has been used in the attached marimo notebooks (Wk02_Marimo_ZScore.py, Wk03_InteractivePlots_Plotly.py, Wk04_DataPreparation.py) to help me understand its usage better.

GitHub Copilot here — concise at-a-glance summary of how pandas is used across the three notebooks.

- Reading / writing
 - Read CSV: `df = pd.read_csv('public/sp500_raw_data_Backup.csv')`
 - Save CSV: `df.to_csv('public/sp500_ZScore_AvgCostofDebt.csv', index=False)`
- Basic inspection & display
 - Show head / display DataFrame: `df.head()` or just `df` (marimo/Jupyter friendly)
 - Unique values: `df['Sector_Key'].unique()`
- Missing-data handling
 - Replace NaNs: `df['Total_Debt'] = df['Total_Debt'].fillna(0)`
 - Drop rows with missing key vars: `df.dropna(subset=['AvgCost_of_Debt', 'Z_Score_lag'])`
 - Tests for NaN: `df[df['Total_Debt'].isna()], df[df['Z_Score'].notna()]`
- Selection / boolean filtering
 - Filter by column value: `df[df['Ticker']=='MSFT']`
 - Compound filters: `df[(df['Z_Score_lag']<50) & (df['AvgCost_of_Debt']<5)]`
- Sorting and copies
 - Ensure chronological order: `df.sort_values(['Ticker','Year'])`
 - Make safe editable copy: `df_msft = df[df['Ticker']=='MSFT'].copy()`
- Vectorized arithmetic / new columns
 - Create columns with vectorized ops (fast, element-wise):
`df['Z_Score'] = Zscore(... df['Total_Assets'] ...)`
`df['avg_debt'] = 0.5*(df['Total_Debt'] + df['prev_debt'])`
`df['Debt_Cost_Percent'] = df['AvgCost_of_Debt'] * 100`
- Lagging / groupwise time ops
 - Per-company lag (shift after sort+groupby):
`df['prev_debt'] = df.sort_values(['Ticker','Year']).groupby('Ticker')['Total_Debt'].shift(1)`
`df['Z_Score_lag'] = df.sort_values(['Ticker','Year']).groupby('Ticker')['Z_Score'].shift(1)`
- Aggregation & summary stats
 - Median: `median_cost = df['AvgCost_of_Debt'].median()`
 - Count / crosstab: `pd.crosstab(index=..., columns=..., margins=True)`
- Element-wise mapping / apply
 - Categorize with lambda / apply for labels:
`tb['Risk_Zone'] = tb['Z_Score_lag'].apply(lambda x: 'Safe' if x>2.99 else ...)`
- Preparing data for modeling / plotting
 - Filter out extreme outliers before regressions/plots:
`reg_df = df_clean[df_clean['AvgCost_of_Debt'] < 5]`

- Use Series as inputs to statsmodels:

```
Y = reg_df['AvgCost_of_Debt']; X = sm.add_constant(reg_df['Z_Score_lag'])
```

- Working with pandas Series from external libs
 - Use `.iloc` to pick a column from yfinance DataFrame: `bs = stock.balance_sheet.iloc[:,0]`
 - Safe lookup on a Series: `bs.get('Total Assets')` (returns None if missing)
- Display & formatting helpers
 - Temporarily change print options: `with pd.option_context('display.max_rows', None): print(df)`
 - Convert types when needed: `.astype(float)` used before numpy regression

Overall patterns to learn:

- Prefer vectorized column ops over row loops for speed and clarity.
- Use sort+groupby+shift for panel (per-entity time) lagged features.
- Handle NaNs explicitly (fillna / dropna) depending on semantic meaning.
- Use `.copy()` to avoid chained-assignment/view pitfalls.
- Use apply/lambda for categorical labeling and `pd.crosstab` for contingency tables.

If you want, I can produce a 1-page cheat-sheet with the most common one-liners from these notebooks.

That would be great

GitHub Copilot — 1-page pandas cheat-sheet (concise, from Wk02 / Wk03 / Wk04)

Quick one-liners and patterns you saw in the notebooks:

```

# Read / write
df = pd.read_csv("public/sp500_raw_data_Backup.csv")
df.to_csv("public/sp500_ZScore_AvgCostofDebt.csv", index=False)

# Inspect / display
df.head()
df.info()
df.describe()
df['Sector_Key'].unique()
# marimo / Jupyter: just put `df` to display nicely
with pd.option_context('display.max_rows', None): print(df)

# Missing data
df['Total_Debt'] = df['Total_Debt'].fillna(0)
df[df['Total_Debt'].isna()]
df.dropna(subset=['AvgCost_of_Debt', 'Z_Score_lag'])

# Selection / boolean filtering
df[df['Ticker'] == 'MSFT']
df[(df['Z_Score_lag'] < 50) & (df['AvgCost_of_Debt'] < 5)]

# Sorting, grouping, safe copy
df = df.sort_values(['Ticker', 'Year'])
df_msft = df[df['Ticker']=='MSFT'].copy()

# Vectorized new columns (fast; avoid row loops)
df['AvgCost_of_Debt'] = df['Interest_Expense'] / df['Total_Debt']
df['Debt_Cost_Percent'] = df['AvgCost_of_Debt'] * 100
# apply custom Zscore function elementwise (works with Series)
df['Z_Score'] = Zscore(df['Total_Assets'], df['Current_Assets'], df['Current_Liab'],
                      df['Retained_Earnings'], df['EBIT'], df['Total_Liab'],
                      df['Sales'], df['Market_Cap'])

# Panel / lag features per entity
df = df.sort_values(['Ticker', 'Year'])
df['Z_Score_lag'] = df.groupby('Ticker')['Z_Score'].shift(1)
df['prev_debt'] = df.groupby('Ticker')['Total_Debt'].shift(1)

# Aggregation / summaries
df.groupby('Year')['AvgCost_of_Debt'].median()
pd.crosstab(df['Year'], df['Sector_Key'], margins=True)

# Apply / map for labels
tb['Risk_Zone'] = tb['Z_Score_lag'].apply(lambda x: 'Safe' if x > 2.99 else ('Distress' if x < 1.81

# Prepare for regression / plotting
reg_df = df.dropna(subset=['AvgCost_of_Debt', 'Z_Score_lag'])
Y = reg_df['AvgCost_of_Debt']
X = sm.add_constant(reg_df['Z_Score_lag'])

```

```
# Helpful practices
# - Prefer vectorized ops over Python loops
# - Always sort by entity + time before groupby().shift()
# - Use .copy() when creating slices you will modify
# - Handle NaNs explicitly with fillna/dropna depending on meaning
```

If you want, I can convert this into a printable one-page PDF or a Markdown file in the repo.